

Assignment-19.1

HT.NO:2303A51732

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

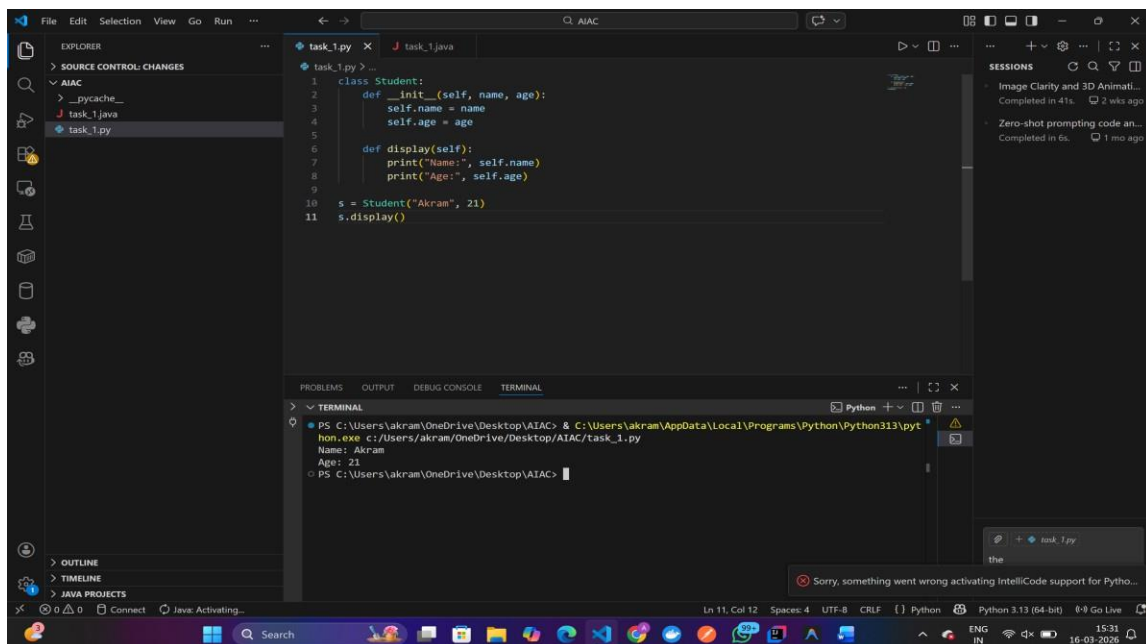
Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

CODE:

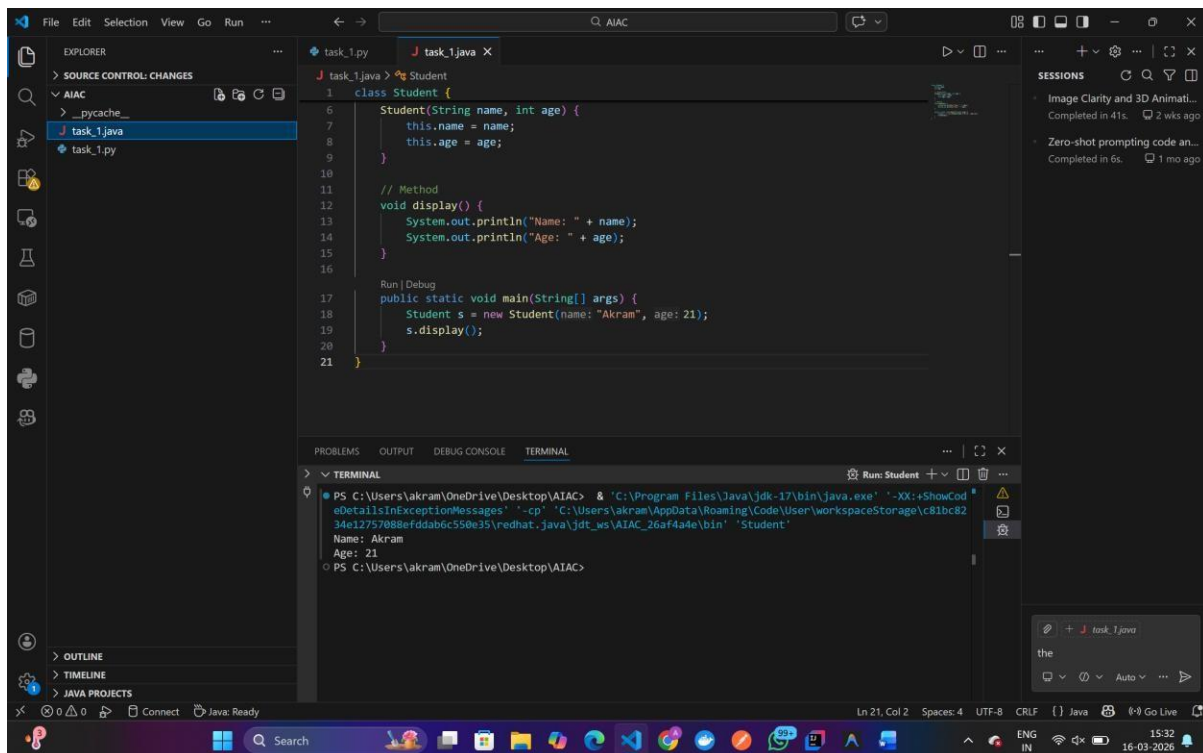


The screenshot shows a code editor with a Python file named `task_1.py`. The code defines a `Student` class with an `__init__` method and a `display` method. An instance `s` is created with the name 'Akram' and age 21, and the `display` method is called. The terminal output shows the execution of the Python script, which prints 'Name: Akram' and 'Age: 21'. A message at the bottom indicates that IntelliCode support for Python is not activated.

```
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def display(self):
7         print("Name:", self.name)
8         print("Age:", self.age)
9
10 s = Student("Akram", 21)
11 s.display()
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

Sorry, something went wrong activating IntelliCode support for Python...



Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

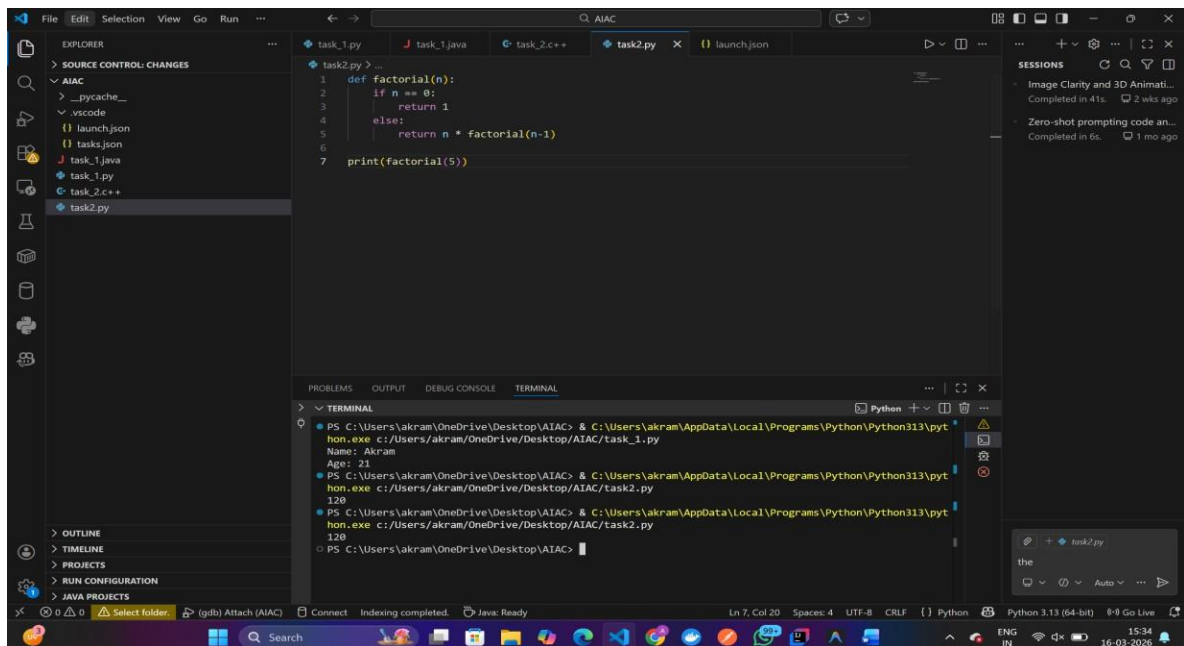
Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

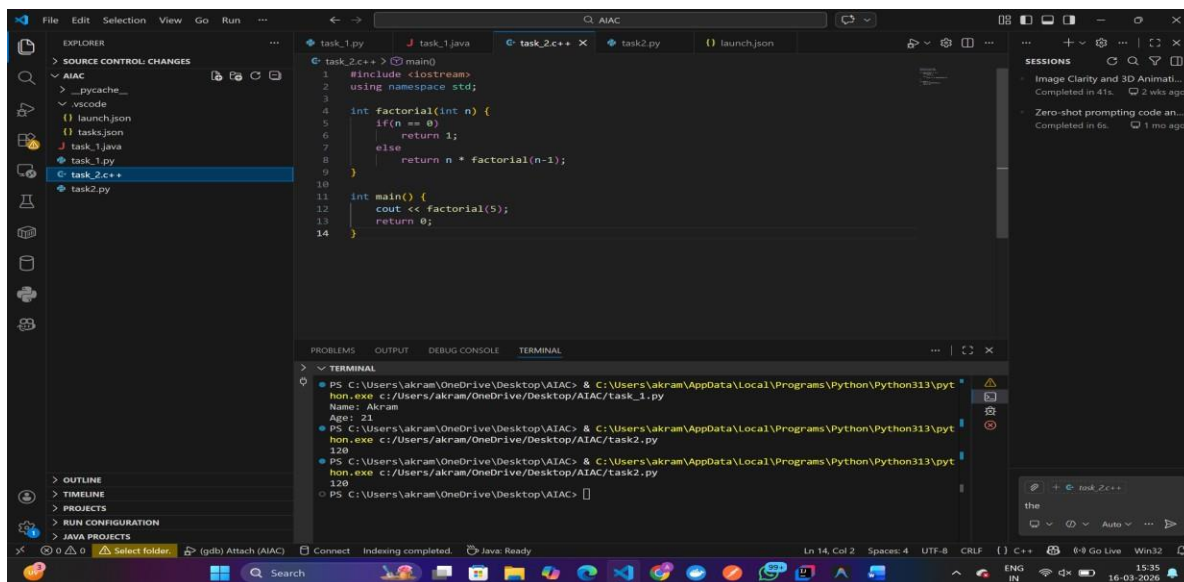
CODE:



```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

print(factorial(5))
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```



```
#include <iostream>
using namespace std;

int factorial(int n) {
    if(n == 0)
        return 1;
    else
        return n * factorial(n-1);
}

int main() {
    cout << factorial(5);
    return 0;
}
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

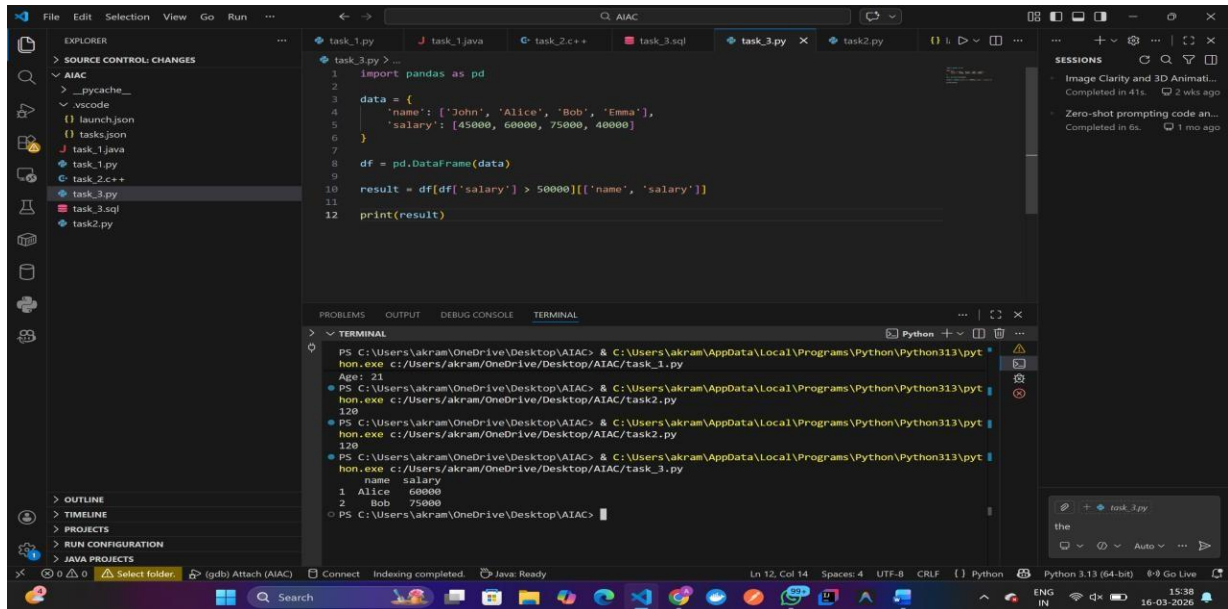
Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

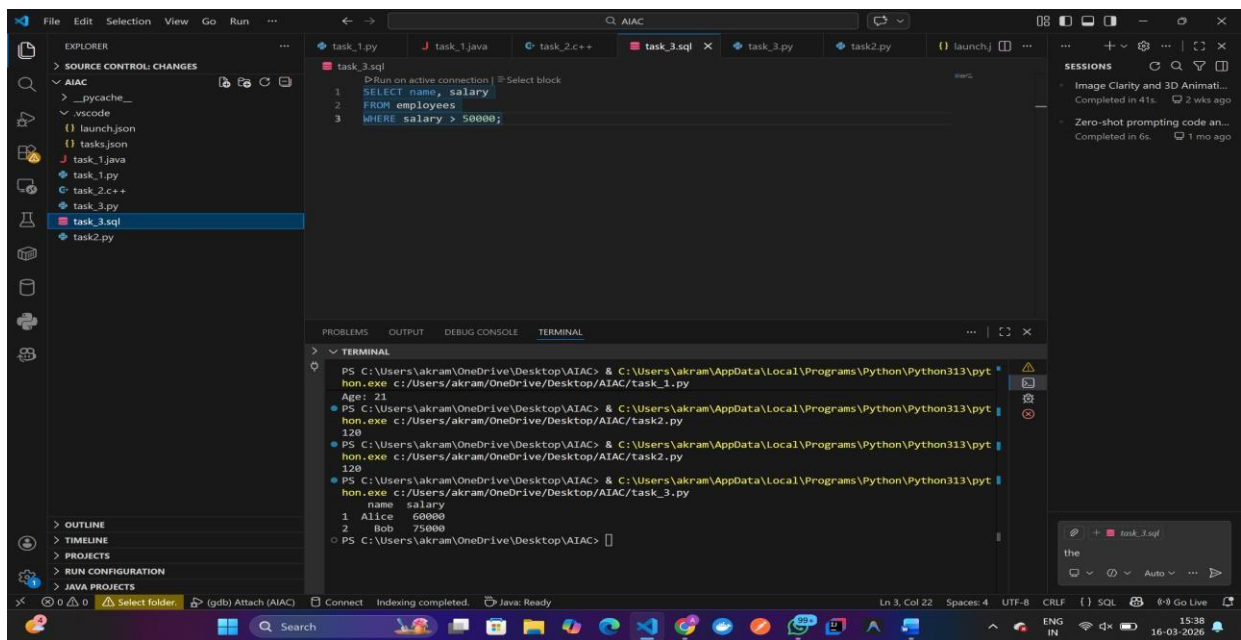
CODE:



The screenshot shows the Visual Studio Code editor with a Python file named `task_3.py` open. The code defines a dictionary `data` with names and salaries, converts it to a pandas DataFrame, and filters for salaries greater than 50,000. The terminal shows the execution of the script, displaying the filtered data.

```
1 import pandas as pd
2
3 data = {
4     'name': ['John', 'Alice', 'Bob', 'Emma'],
5     'salary': [45000, 60000, 75000, 40000]
6 }
7
8 df = pd.DataFrame(data)
9
10 result = df[df['salary'] > 50000][['name', 'salary']]
11
12 print(result)
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_3.py
   name  salary
1  Alice   60000
2    Bob   75000
```



The screenshot shows the Visual Studio Code editor with a SQL file named `task_3.sql` open. The code contains a SQL query to select names and salaries from an `employees` table where the salary is greater than 50,000. The terminal shows the execution of the script, displaying the filtered data.

```
1 SELECT name, salary
2 FROM employees
3 WHERE salary > 50000;
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_3.py
   name  salary
1  Alice   60000
2    Bob   75000
```

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.

- Translating a Java method into C# for enterprise applications.

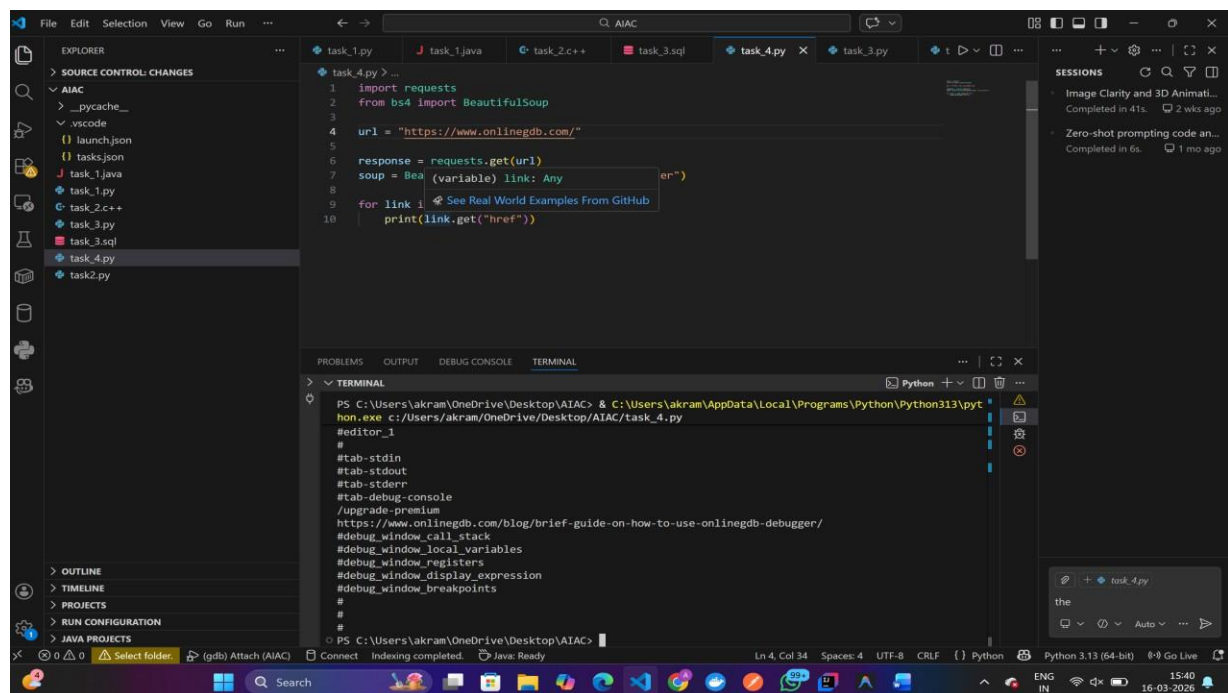
Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages

CODE:



The screenshot shows a Visual Studio Code editor with a Python file named `task_4.py` open. The code is a web scraping script using `requests` and `BeautifulSoup` to fetch and print links from a webpage. The terminal at the bottom shows the command to run the script and its output, which includes a list of links from the OnlineDB website.

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = "https://www.onlinedb.com/"
5
6 response = requests.get(url)
7 soup = BeautifulSoup(response.text, 'html.parser')
8
9 for link in soup.find_all('a'):
10     print(link.get('href'))
```

The terminal output shows the command `python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_4.py` being executed, and the output is a list of links from the OnlineDB website, including `https://www.onlinedb.com/blog/brief-guide-on-how-to-use-onlinedb-debugger/`.

